Questions on BFS and DFS

[3 points] For a directed graph, the absence of back edges with respect to a BFS tree implies that the graph is acyclic.

Solution: FALSE. It is true that the absence of back edges with respect to a *DFS* tree implies that the graph is acyclic. However, the same is not true for a BFS tree. There may be cross edges which go from one branch of the BFS tree to a lower level of another branch of the BFS tree. It is possible to construct a cycle using such cross edges (which decrease the level) and using forward edges (which increase the level).

[3 points] The depth of any DFS tree rooted at a vertex is at least as much as the depth of any BFS tree rooted at the same vertex.

Solution: TRUE. Since BFS finds paths using the fewest number of edges, the BFS depth of any vertex is at least as small as the DFS depth of the same vertex. Thus, the DFS tree has a greater or equal depth.

[3 points] There is no edge in an undirected graph that jumps more than one level of any BFS tree of the graph.

Solution: TRUE. If such an edge existed, it would provide a shorter path to some node than the path found by BFS (in terms in the number of edges). This cannot happen, as BFS always finds the path with the fewest edges.

[3 points] BFS takes $O(V + E)$ time irrespective of whether the graph is presented with an adjacency list or with an adjacency matrix.

Solution: FALSE. With an adjacency matrix representation, visiting each vertex takes $O(V)$ time, as we must check all N possible outgoing edges in the adjacency matrix. Thus, BFS will take $O(V^2)$ time using an adjacency matrix.

[3 points] An undirected graph is said to be *Hamiltonian* if it has a cycle containing all the vertices. Any DFS tree on a Hamiltonian graph must have depth $V - 1$.

Solution: FALSE. If a graph has a Hamiltonian cycle, then it is *possible*, depending on the ordering of the graph, that DFS will find that cycle and that the DFS tree will have depth $V - 1$. However, DFS is not guaranteed to find that cycle. (Indeed, finding a Hamiltonian cycle in a graph is NP-complete.) If DFS does not find that cycle, then the depth of the DFS tree will be less than $V - 1$.

Suppose we do a DFS on a directed graph G. If we remove all of the back edges found, the resulting graph is now acyclic.
Explain:

Solution: **True.** If DFS finds no back edges, then the graph is acyclic. Removing any back edges found doesn't change the traversal order of DFS, so running DFS again on the modified graph would produce no back edges.

A depth-first search of a directed graph always produces the same number of tree edges (i.e. independent of the order in which the vertices are provided and independent of the order of the adjacency lists).
Explain:

Solution: **False.** The DFS forest may contain different numbers of trees (and tree edges) depending on the starting vertex and upon the order in which vertices are searched.

Both DFS and BFS require $\Omega(V)$ storage for their operation. (That is, for working storage, above and beyond the storage needed to represent the input.)
Explain:

Solution: **True.** Each needs to keep track of the vertices that have already been visited.

If a directed graph G is cyclic but can be made acyclic by removing one edge, then a depth-first search in G will encounter exactly one back edge.

Solution: False. You can have multiple back edges, yet it can be possible to remove one edge that destroys all cycles. For example, in graph $G = (V, E) = (\{a, b, c\}, \{(a, b), (b, c), (b, a), (c, a)\})$, there are two cycles ($[a, b, a]$ and $[a, b, c, a]$) and a DFS from a in G returns two back edges ((b, a) and (c, a)), but a single removal of edge (a, b) can disrupt both cycles, making the resulting graph acyclic.

Give a counterexample to the conjecture that if there is a path from u to v in a directed graph G, and if d[u] < d[v] in a depth-first search of G, then v is a descendant of u in the depth-first forest produced.

Give a counterexample to the conjecture that if there is a path from u to v in a directed graph G, then any depth-first search must result in d[v] < f[u].

Solution:

